

Stage 1: Application Foundation [complete]

At the end of this stage a user will be able to:

1. Download and install the application on Windows or Mac
2. Open the application and see the main window with menu structure
3. View an image in the main viewport with pan and zoom
4. View images in a configurable grid of up to 2x3 viewports, allowing side-by-side comparison of up to 6 images
5. Pan and zoom within each viewport
6. Navigate between tools via a structured menu/toolbar layout

Behind the scenes:

1. Electron application scaffold with cross-platform build pipeline (the framework that lets the app run on both Windows and Mac from a single codebase)
2. Automated builds and GitHub releases via CI (new versions are built and published automatically when I push updates)

Stage 2: Core Data Pipeline

At the end of this stage a user will be able to:

1. Load single images and image stacks (.tif, .png, .jpg, .DNG, .CR3, and other raw camera formats)
 1. Emphasize all standard image formats, and raw formats where possible.
2. Load and save data in ENVI format or image stack.
 1. Option to bit shift or otherwise work with strangely formatted images since many multispectral cameras pack 12-bit recordings into 16-bit/2-byte containers, so may need to manually shift by 4 bits etc.
3. Save images in multiple formats and bit depths (8-bit, 12-bit, 16-bit)
4. Save and resume project state
5. Spatially crop images and *manage bands* in a stack (keep/remove)
6. Select rectangular regions for ROIs or as input to other tools
7. Inspect pixel values and view spectra (with averaging) for selected pixels
8. View image metadata (format, dimensions, band count) in a separate box

Stage 3: Normalization & Pre-Processing

At the end of this stage a user will be able to:

1. *Apply flat-field correction using a set of reference images.*
 1. Given a stack of images R and an equal-size stack of flat fields F (white reference), optional equal-size stack of dark frames D (set to 0 if none is given), this produces the following stack (via pointwise addition/subtraction and multiplication/division)

$$C = \frac{m(R-D)}{F-D},$$

Where m is the average value of each band of $(F - D)$. So, m is a b -dimensional vector, where b is the number of bands, and multiplies each band of $R - D$.

Python code:

```
def flatField(raw, bright, dark=None):
    """
    raw, bright, and dark should be arrays of size (m,n,b),
    where b is the number of bands
    """
    assert raw.shape == bright.shape
    if dark is None:
        dark = np.zeros(raw.shape)
        #could also compute separately here for efficiency
        #implemented this way for simplicity at the moment
    else:
        assert dark.shape == raw.shape
    flatDifference = bright - dark
    assert np.sum(flatDifference == 0) == 0
    m = np.mean(flatDifference, axis =
    (0,1)).reshape(1,1,raw.shape[2])
    return (raw-dark)*m/(flatDifference)
```

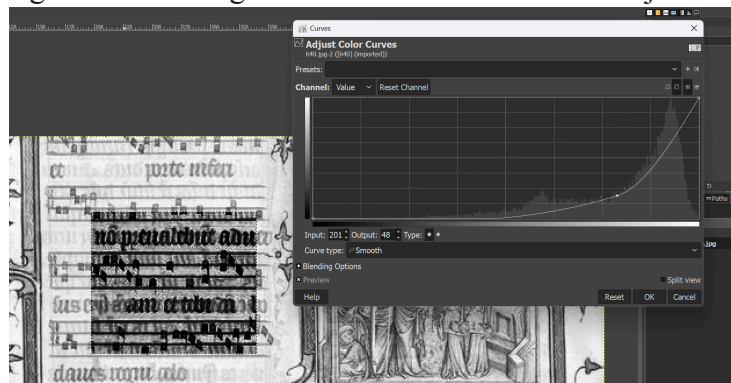
2. *Select a Spectralon reference as reflectance to max value across all bands or similarly scale bands according to a calibration flow.*

1. First, users select a ROI for bright pixels, and optionally for dark pixels.

2. Then, across each band compute the mean in that ROI of bright pixels F and dark pixels D (each being a b -dimension vector). Replace the raw data with $\frac{raw-D}{W-D} * constant$, where *constant* is an optional known reflectance value (default set to 1) of the spectralon panel (to get true physical reflectance measurements)

3. *Adjust contrast via a histogram interface (whole image or selected region)*

1. For the histogram interface, refer to GIMP, where in entire image or selected region has a histogram with a curve that can be adjusted as the image below:



4. *Adjust brightness/contrast with sliders or input values:*

1. Implement brightness shift by adding or subtracting some value to an image. So for a parameter α replace an image I with $I + \alpha$.

2. For contrast, multiply around the mean, i.e. replace an image I $(I - \text{mean}(I)) * \beta + \text{mean}(I)$
3. In both cases, after the transformation clip to same data type of image. For example, with an 8-bit image, everything above 255 gets sent to 255.
5. *Invert images or selected bands*
 1. Here this does not mean $-1 * I$, but instead the max of the image data type minus the image. In other words, for an 8-bit image I , this is $255 - I$.
 2. If the data type is not known (e.g. after a PCA transform), first normalize to lie in (0,1) and then take 1 minus the image. In that case, may be good to return both the normalized, and normalized + inverted image.
6. *Normalize images (min to 0, max to 1) linearly*
 1. In general, an array A is replaced by $(A - (A)_{\min}) / ((A)_{\max} - (A)_{\min})$
 2. There should be a choice to apply to the whole cube or band-wise (to take min/max over cube, or min/max per band). To normalize a subset of bands, those bands can be selected and then the normalization can be applied.
7. *Set target means and standard deviations for standardization*
 1. In general, an array A is replaced by $\frac{A - \text{mean}}{\text{std}} * \text{desired std} + \text{desired mean}$
 2. As with *Normalize Images*, can be applied by band or to cube as a whole (e.g transforming the cube mean and standard deviation or band-wise mean and standard deviation), or to selected images.
8. *Convert RGB to grayscale*
 - a. Standard R/G/B weights of 0.2989, 0.5870, 0.1140 by default with option to add custom weights, e.g. 1/3, 1/3, 1/3
9. *Create false-color composites by selecting 3 bands from a stack*
 1. Allow selection of 3 bands to become RGB

Stage 4: Core Dimension Reduction & Analysis

At the end of this stage a user will be able to:

1. *Compute PCA on full image (default) or a selected ROI, choose number components to compute, and apply to the full image*
 1. Python implementations if helpful:
 - a. <https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.PCA.html>
 - b. https://www.spectralpython.net/class_func_ref.html#spectral.principal_components
 2. Apply PCA to a selected ROI means the PCA is 'fitted' to the ROI and the transformation is applied to the entire image area.
2. *MNF*
 1. Python implementation if helpful:
 - a. https://www.spectralpython.net/class_func_ref.html#minimum-noise-fraction-mnf
 2. Similar ROI or full image as with PCA, and similarly input number of components to keep (number of bands in output image).
3. *ICA*
 1. Python implementation if helpful:

a. <https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.FastICA.html>

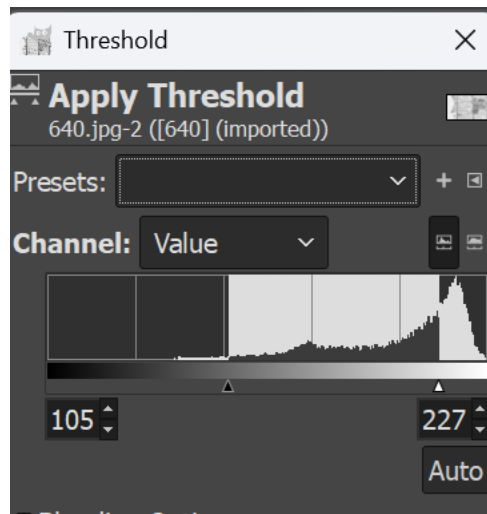
2. Similar ROI or full image as with PCA and MNF, and similarly input number of components to keep (number of bands in output image).

Stage 5: Filters & Multi-Image Operations

At the end of this stage a user will be able to:

1. *Threshold images manually or with Otsu.* ~~Also Niblack and Sauvola.~~

1. For manual thresholding, could this be in GIMP style as below, with an upper and lower bound?



2. For Otsu thresholding, this will automatically determine the cutoff for white/black.

~~2. Subtract one image from another with optional normalization, resulting in another image.~~

3. *Derivative of image in spectral dimension.*

1. Should output a cube of the same dimension as the input.

2. Can be first-order approximation (may be useful to implement after weight bands, since here every new band is a weighting of bands near it), or second order approximation.

4. *Apply high-pass, low-pass, and bandpass filters*

1. See e.g.

https://scikit-image.org/docs/0.25.x/auto_examples/filters/plot_butterworth.html

5. *Denoising/Outlier Removal:*

1. Gaussian filter (i.e. convolve by Gaussian kernel) with kernel size input.

2. Median filter, replacing each pixel by the median of a surrounding window, with window size as an input.

~~3. Remove row-based noise from HHS data with configurable sensitivity~~

4. Clip top and bottom percent of values.

- a. Give a lower bound and upper bound percentile. Then, clip everything above the upper bound percentile value to that value and same with the lower bound, i.e.

`lb = np.percentile(data, lower)`

`ub = np.percentile(data, upper)`

`result = np.clip(data, lb, ub)`

- b. This should be applicable per band or over the entire cube. In other words, the lower/upper bounds can be computed over the entire cube or band-by-band.

6. Linear Combination (Weighting) of Bands

1. User defined linear combination of bands.

2. Give each band a weight, $w_1, \dots, w_b$. Then normalize the weights by $s = \sum |w_i|$

(ensure $s > 0$) so that $w'_i = w_i/s$. Then output an image that is

$$I = \sum w'_i * band_i.$$

3. The user can manually input weights. For ease of the user, it should be possible to reset weights everything to 0, or reset all weights to 1.

4. There can also be automatic weights, given by NPC or some other function.

7. Selections bands via some function

1. Select bands using some evaluation function, like NPC. Could be something else, or could be a user-defined input range of bands to keep.

8. Rotation by 90, 180, 270 degrees

1. Rotate the entire cube to ensure that all spatial dimensions are the same.

9. Linked Pan and Zoom

1. Pair viewing ports so they pan and zoom in sync. The viewing ports should have the same spatial dimensions, and show the same spatial locations (rather than independent navigation)
2. That way, we can see changes by investigating the same details easily.

For 6 and 7, a question: is it possible to input a custom function from the user, and if so how? In python syntax? Or would we need a predefined list of simple functions, e.g. mean of the band, variance of the band, etc. This also applies to point 4 in Stage 6.

Maybe this fits in with adding custom scripts?

Stage 5.5 (can be done at any time): Spawn Python worker for custom scripts.

Stage 6: Advanced Tools & Masking

At the end of this stage a user will be able to:

1. *Create and manage (save, delete, edit) multiple mask layers with labeled regions (foreground, background, etc.).*

1. We might need to think a little bit more about how to manage masks. Perhaps visualized via additional ‘mask layers’ on a stack or similar, perhaps can be darker or lighter where annotated?
2. *Concatenate two different cubes*
 1. Allow the concatenation of bands from two different cubes, ensuring they have the same spatial dimension.
 - a. This is very useful for when two datasets with the same camera are captured e.g. under UV light and Infrared light.
3. *Similar to point 2, would be useful here to allow duplication of selected bands, i.e. concatenate a selected band to a cube.*
4. *Compute ROP (random orthogonal projections) with new random visualizations on each press*
 1. Will provide our ROP code later.
5. *Compute NPC with Masks*
 1. <https://github.com/wallacepeaslee/Multiple-Class-Normalized-Potential-Co-contrast> (Note: may have bug with negative or strangely-normalized data).
6. *Use Multi-Class NPC as the optimization function for ROP*
 1. Variance, other customized functions later. See note on custom functions from stage 5.
7. *Compute spatially-localized/adaptive PCA and MNF with configurable stride, kernel width, and intensity*
 1. Code to be provided later.
8. *L2-minimization, requiring user masks and some input parameters*
 1. Code to be provided later.

Wish list in priority order(if we finish the other stages early):

- Compute no-reference image quality assessment (nrIQA).
- Unsharp mask.
- Apply radial undistortion given a calibration grid image.
- Apply homography transformations to align images taken from different angles.
- Register and mosaic multiple overlapping image stacks into a single combined stack.
- Compute Fisher Linear Discriminant analysis.
- Compute SpectralIAM (spectral angles).